

Highlights

Efficient and explainable multivariate workload prediction with sparse models

Simone Tonini, Giuseppe Squillace, Francesca Chiaromonte, Mirco Tribastone, Andrea Vandin

- Efficient workload prediction via sparse, interpretable models
- Mitigation of spurious dependencies in workload modeling
- Performance improvement in workload forecasting: accuracy and sparsity

Efficient and explainable multivariate workload prediction with sparse models[★]

Simone Tonini^a, Giuseppe Squillace^b, Francesca Chiaromonte^{a,c}, Mirco Tribastone^b and Andrea Vandin^{a,d,*}

^a*Sant'Anna School for Advanced Studies, Pisa, Italy*

^b*IMT School for Advanced Studies Lucca, Lucca, Italy*

^c*The Pennsylvania State University, Pennsylvania, USA*

^d*DTU Technical University of Denmark, Copenhagen, Denmark,*

ARTICLE INFO

Keywords:

Workload prediction
Regularization
LASSO
Serial correlation
Performance analysis

ABSTRACT

Accurate workload prediction is essential for efficient resource management in operational telemetry. While deep learning methods have become the de-facto standard in this domain, they often entail high computational costs and limited interpretability, restricting their applicability. In this paper, we show that sparse regularized methods provide a competitive and principled alternative. We analyze the impact of serial correlation on LASSO-based sparse vector autoregressive (VAR) models in multivariate settings, finding that it causes unstable variable selection and misleading dependency structures. We solve this issue by adopting ARMAR-LASSO, a sparse VAR estimator which performs pre-whitening of the predictors through ARMA filtering. We support our analysis with illustrative examples, simulation experiments, and an evaluation on benchmark datasets. Across fixed-sample, rolling-window, and multi-step prediction settings, ARMAR-LASSO consistently improves prediction accuracy while selecting fewer predictors than standard LASSO. Moreover, it outperforms state-of-the-art deep learning approaches at a fraction of the computational cost, while yielding parsimonious, stable and interpretable results in workload prediction for complex software systems.

1. Introduction

Predicting the evolution of dynamic workloads has become a fundamental requirement for the efficiency of modern software systems. In cloud-native and microservice-based environments, timely anticipation of workload fluctuations enables proactive resource management decisions, such as dynamic scaling [14, 24], scheduling [70, 2, 37], and load balancing [49], which are crucial to ensure performance, reliability, and cost efficiency.

Accurate workload predictions allow systems to react before performance degradation occurs, reducing overprovisioning and unnecessary energy [34]. As software systems continue to grow in scale and complexity, the ability to predict workload dynamics in an accurate, efficient, and interpretable manner is increasingly critical for performance engineering and operational decision-making [47, 44].

In practice, future workload dynamics are commonly inferred from historical observations stored as time-series.

[★]The work has been partially supported by Tuscany X.0 - Toscana Digital Innovation HUB, funded by the European Union's Digital Europe Programme (DIGITAL) under grant agreement N. 101083396, and by the Italian Department for Digital Transformation (Presidency of the Council of Ministers), under the pilot initiative #REG4IA - 'Hub/Centri regionali I.A. per Resilienza e Sicurezza del Territorio+5G' led by the Tuscany Region.

*Corresponding author

 simone.tonini@santannapisa.it (S. Tonini);

giuseppe.squillace@imtlucca.it (G. Squillace); fxcl1@psu.edu (F.

Chiaromonte); mirco.tribastone@imtlucca.it (M. Tribastone);

andrea.vandin@santannapisa.it (A. Vandin)

ORCID(S): 0000-0002-6325-9533 (S. Tonini); 0000-0003-4004-6049 (G. Squillace); 0000-0001-5605-9886 (F. Chiaromonte); 0000-0002-6018-5989 (M. Tribastone); 0000-0002-2606-7241 (A. Vandin)

Software applications typically exhibit highly variable workloads during execution, often characterized by recurring patterns and bursts [47]. Workload prediction techniques exploit these temporal regularities to infer future behavior from past observations. Time-series analysis has been extensively studied across a wide range of application domains, including economics [27], finance [61], transportation [20], and workload balance [33], just to name a few. In recent years, significant methodological advances have been proposed, particularly in the context of machine learning and deep learning [68]. However, these methods often exhibit limitations in terms of interpretability, and computational cost [46].

Motivated by these limitations, this work focuses on workload prediction in complex, distributed software systems and investigates regularized methods as a principled alternative to black-box predictors. Following [69] and the Occam's razor principle, we emphasize *simple* parsimonious and interpretable approaches that can effectively capture cross-metric dependencies and temporal dynamics while remaining computationally efficient, robust, and interpretable. In particular, we adopt a novel multivariate method for workload prediction based on the Least Absolute Shrinkage and Selection Operator (LASSO) [58], a regularized regression method widely adopted in statistics, econometrics, and statistical learning [31]. LASSO performs estimation and variable selection simultaneously by imposing a penalty, yielding sparse models that mitigate overfitting and enhance generalization performance. The induced sparsity improves model interpretability, reduces variance, and facilitates robustness in high-dimensional settings where the number of

predictors may be large relative to the sample size. Moreover, LASSO solutions can be computed efficiently using convex optimization techniques, making the approach scalable and practical for large datasets.

Despite these advantages, LASSO-based methods can behave poorly in the presence of strong correlations among predictors. In operational telemetry data, such correlations arise not only from genuine cross-metric interactions but also from serial dependence, which reduces the effective number of independent observations and can substantially distort variable selection. In particular, the presence of serial correlation may lead LASSO to select spurious variables, resulting in unstable models and misleading dependency structures. This issue is especially problematic in multivariate workload modeling, where sparse coefficients are often interpreted as directed dependency graphs and false positives can mislead performance diagnosis and capacity-planning decisions.

While it is well known that serial dependence inflates sample correlations and may induce spurious variable selection, a less emphasized issue arises when temporal dependence is not fully captured by the model. In such cases, persistence propagates through the system and alters the population covariance structure, so that lagged variables may exhibit non-zero cross-correlations even in the absence of direct links in the VAR coefficient matrices (see, e.g., [27] and [43]). This phenomenon is particularly relevant in sparse multivariate settings, where non-zero coefficients are often interpreted as cross-metric dependencies, and may therefore reflect persistence-driven associations rather than genuine structural relationships.

Recent work on ARMAr-LASSO [59] has shown that, in univariate regression settings, serial correlation can induce spurious associations that degrade LASSO's selection accuracy, and that pre-whitening can mitigate this effect. However, its implications for dependency estimation in multivariate sparse VAR models are less understood. This can undermine both predictive performance and interpretability, making it important to properly account for serial dependence in workload modeling for software systems.

Contribution In this paper, we investigate the impact of serial correlation on sparse multivariate VAR models for workload prediction in complex software systems. Building on the ARMAr-LASSO framework [59], we extend the idea of pre-whitening to multivariate settings, adapting it to sparse VAR estimation.

Our contributions are threefold. First, we provide empirical evidence that simple sparse linear models can match or outperform state-of-the-art deep learning approaches in multivariate workload prediction, while being substantially more efficient and interpretable. Across multiple datasets and evaluation settings, regularized linear models achieve competitive or superior predictive performance at a fraction of the computational cost. Second, we show that serial correlation can significantly distort dependency estimation in sparse VAR models. In particular, when serial dependence

is not fully captured by the VAR structure, persistence can induce cross-metric associations that arise at the population level, rather than being purely a finite-sample artifact. This effect can lead to misleading dependency structures and unstable variable selection. Third, we demonstrate that pre-whitening the predictors via ARMA filtering effectively mitigates these issues in multivariate settings, improving both prediction accuracy and the stability and sparsity of the selected models. Taken together, our results highlight the importance of properly accounting for serial dependence in multivariate workload modeling and show that simple, interpretable methods can provide a competitive alternative to more complex deep learning approaches.

2. Related Work

Early methods Early workload forecasting methods relied on classical time-series models, such as ARIMA and its extension DEC-ARIMA, to improve the load prediction in SLA-aware (Service Level Agreement) VM-scaling systems [4, 35, 38]. In [10], ARIMA is employed for dynamic resource provisioning, presenting limitations in predicting the peak resource utilization. Although these approaches provide strong theoretical foundations and high interpretability, they often struggle with workload prediction, where the dynamics are characterized by bursts of charge [56]. Multivariate extensions based on VAR modeling allow joint forecasting of multiple resource metrics. Nevertheless, as dimensionality increases, VAR models become prone to overparameterization and multicollinearity [18].

Traditional machine learning methods Traditional machine learning techniques aim to overcome the limitations of purely statistical models in capturing non-linear workload dynamics. Approaches based on Support Vector Machines (SVM) and Random Forests have been widely explored due to their flexibility and strong empirical performance. For instance, Barati et al. [5] proposed an SVM-based prediction framework tuned using a hybrid Genetic Algorithm and Particle Swarm Optimization. Similarly, Zhong et al. [72] introduced a wavelet-enhanced SVM model, combining signal decomposition with evolutionary optimization techniques to improve prediction accuracy. In parallel, Cetinski and Juric [11] investigated schemes based on Random Forest for both classification and regression tasks in workload prediction.

Deep learning methods The success of these machine learning approaches encouraged a shift toward deep learning techniques, particularly neural network architectures capable of modeling complex temporal patterns and non-linear relationships. These models offer enhanced expressive power but typically at the cost of increased training complexity and reduced interpretability. Notable examples include hybrid models that integrate neural networks with classical predictors, such as ARIMA [19] and SVM [57], as well as architectures leveraging autoencoders for feature extraction and dimensionality reduction [38, 13].

Recurrent Neural Networks (RNNs) were subsequently adopted due to their natural suitability for sequential data. However, standard RNNs struggle to capture long-term dependencies, a limitation later addressed by the introduction of Long Short-Term Memory (LSTM) networks. For example, Tuli et al. [62] employed LSTM models to improve SLA-aware resource management, reporting reductions in SLA violations. Patel et al. [52] presented a case study showing that LSTM architectures can outperform convolutional neural networks in VM workload prediction. Gupta et al. [26] highlighted the benefits of multivariate LSTM models, demonstrating that jointly modeling multiple resource metrics can outperform univariate formulations.

More recently, research has explored increasingly sophisticated architectures, including convolutional and hybrid models based on convolutional neural networks designed to capture multi-scale temporal dependencies and long-range correlations [64, 65, 39]. Transformer-based models, leveraging self-attention mechanisms, have further advanced time-series forecasting by enabling adaptive weighting of historical observations [73, 74, 66].

Alongside these general-purpose architectures, recent work has also proposed domain-specific forecasting solutions tailored to particular systems, workloads, or architectural settings [41, 42, 22, 23, 40, 67]. These approaches are highly relevant, as they exploit structural properties of the target domain to improve forecasting accuracy. However, they are typically designed for specific scenarios and, to the best of our knowledge, do not provide readily available implementations that can be directly integrated into a common benchmarking pipeline. Since our study adopts a practitioner-oriented perspective and focuses on reproducible methods that are easy to deploy with standard forecasting libraries, these domain-specific approaches are outside the scope of our empirical comparison.

Our work Interestingly, recent findings suggest that simpler architectures, such as linear models and multilayer perceptrons (MLPs), can compete or even outperform more complex deep learning models in several long-term forecasting benchmarks [16, 21, 69]. Similar results have been found also in other domains such as anomaly detection [55, 60]. These observations challenge the assumption that architectural complexity necessarily translates into superior predictive performance. Our work aligns with this perspective by demonstrating that simple and parsimonious models can provide competitive, if not superior, results in multivariate workload prediction. In particular, we show that regularized statistical models, when properly adjusted to account for serial correlation, offer a robust, explainable, and computationally efficient alternative to black-box deep learning approaches.

3. Background

Modern software systems generate large volumes of operational telemetry, such as CPU utilization, memory consumption, disk and network I/O, and service-level latency.

These metrics are typically recorded at fine time granularity and exhibit two recurring properties: (i) strong serial dependence (e.g., diurnal/weekly patterns and persistence), and (ii) substantial cross-correlation across metrics (e.g., CPU and latency co-move under load, and I/O and memory pressure interact). These characteristics complicate both *forecasting* and *interpretation*: models that fit well may still yield unstable or misleading dependency structures, which in turn can affect capacity planning, anomaly triage, and performance diagnosis [1].

A common approach to multivariate time-series modeling is to represent the evolution of the metric vector through linear dynamical models, including Vector Autoregressions (VAR) (see, e.g., [27], ch. 11), which explain each metric as a function of lagged values of multiple metrics. In operational settings, however, the number of candidate predictors grows quickly with the number of metrics and lags, often leading to high-dimensional regression problems. However, here, fortunately, one may expect that only a subset of lagged interactions is expected to be actually relevant. This has motivated the adoption of *regularized regression* methods for time-series analysis, where a penalty term is added to the loss function to control model complexity and improve generalization (see, e.g., [58, 7, 8]).

Among these methods, the Least Absolute Shrinkage and Selection Operator (LASSO)[31] is widely used because it simultaneously performs shrinkage and variable selection by adding an ℓ_1 -penalty to the least-squares objective. Intuitively, the ℓ_1 penalty encourages many coefficients to become exactly zero, producing sparse models that are easier to interpret and often more robust when predictors are numerous [58]. This property is attractive in software and systems contexts, where practitioners frequently seek compact dependency graphs (e.g., “which metrics predict which others at short horizons?”) and parsimonious forecasting models.

Despite its practical appeal, it is well known that LASSO can behave poorly when predictors are strongly correlated [71]. In such cases, coefficient estimates may become unstable, selection may be inconsistent, and the method may include spurious variables or arbitrarily choose one predictor from a group of highly correlated candidates [71, 8, 51]. These issues are especially relevant for telemetry data, where (cross-)correlations are pervasive [48].

A less emphasized, but critical, complication in operational time-series is that *serial correlation* can amplify apparent linear dependence among predictors in finite samples. This has been recently studied in univariate settings in [59], where “univariate” refers to the presence of a single target variable (as opposed to multiple dependent variables), even though the predictor space may be multivariate. Even when predictors are weakly correlated at the population level, temporal persistence reduces the effective number of independent observations and can induce non-negligible *spurious correlations*, increasing the risk of false discoveries in variable selection. This undermines the reliability of dependency structures inferred from regularized models

and can lead to misleading conclusions about cross-metric interactions.

This paper addresses these challenges by focusing on sparse multivariate workload modeling under serial dependence. In particular, building on recent evidence that removing serial dependence can stabilize LASSO-based selection in univariate time-series settings [59], we develop and evaluate an approach that reduces spurious dependencies while preserving model interpretability—a central requirement in systems and software engineering tasks.

4. Methodology

This section describes our multivariate workload modeling framework based on sparse VAR models and introduces the proposed ARMAR-LASSO estimator. We then explain why serial correlation can inflate linear dependence among lagged predictors, leading to spurious edges in inferred dependency graphs, and how pre-whitening mitigates this effect while preserving interpretability. Throughout the section, we make use of an *illustrative example* that accompanies the different subsections and helps clarify the mechanisms underlying the proposed approach.

4.1. Sparse VAR workload modeling for system telemetry

Let $\mathbf{y}_t = (y_{1,t}, y_{2,t}, \dots, y_{n,t})'$ denote an n -dimensional (e.g., CPU load, RAM load, etc), zero-mean of system metrics observed over $t = 1, \dots, T$. In practice, each time series is centered by subtracting its empirical mean before estimation, so that the zero-mean assumption is satisfied. We model joint dynamics through a p th-order Vector Autoregression (VAR):

$$\mathbf{y}_t = \mathbf{A}_1 \mathbf{y}_{t-1} + \mathbf{A}_2 \mathbf{y}_{t-2} + \dots + \mathbf{A}_p \mathbf{y}_{t-p} + \boldsymbol{\epsilon}_t, \quad (1)$$

where $\mathbf{A}_\ell$ ($\ell = 1, \dots, p$) are $n \times n$ coefficient matrices and $\boldsymbol{\epsilon}_t$ is an error term. In monitored software systems, the number of candidate predictors grows quickly with the number of metrics and lags, while only a subset of cross-metric effects is typically meaningful. We therefore adopt a *sparsity* perspective, aiming to estimate VAR coefficient matrices with many zeros, yielding models that are both parsimonious and interpretable.

For each target metric $i \in \{1, \dots, n\}$, define the stacked predictor vector $\mathbf{z}_t = (\mathbf{y}'_{t-1}, \dots, \mathbf{y}'_{t-p})'$ (dimension $np \times 1$), and the design matrix $\mathbf{X} = (\mathbf{z}_1, \dots, \mathbf{z}_T)$ (dimension $np \times T$). Let $\mathbf{y}_i = (y_{i,1}, \dots, y_{i,T})'$ be the response vector for equation i , and let $\boldsymbol{\alpha}_i^*$ collect the np regression coefficients for that equation. Then Eq. (1) can be written equation-wise as

$$\mathbf{y}_i = \mathbf{X}' \boldsymbol{\alpha}_i^* + \boldsymbol{\epsilon}_i, \quad i = 1, \dots, n. \quad (2)$$

We assume $\boldsymbol{\alpha}_i^*$ is sparse, i.e., $\|\boldsymbol{\alpha}_i^*\|_0 = s_i < np$.

Example 1. Consider a VAR(1) system:

$$\mathbf{y}_t = \mathbf{A} \mathbf{y}_{t-1} + \boldsymbol{\epsilon}_t, \quad (3)$$

with

$$\mathbf{y}_t = \begin{pmatrix} y_{1,t} \\ y_{2,t} \\ y_{3,t} \\ y_{4,t} \end{pmatrix}, \quad \mathbf{A} = \begin{pmatrix} 0 & \alpha & 0 & \alpha \\ 0 & 0 & 0 & 0 \\ \alpha & \alpha & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$\epsilon_{i,t} = \phi \epsilon_{i,t-1} + \omega_{i,t}, \quad i = 1, \dots, 4.$$

where $\omega_{i,t}$ are mutually uncorrelated white noise processes with variance σ_i^2 .

We focus on the first equation, where the target is $y_{1,t}$. The relevant predictors are $y_{2,t-1}$ and $y_{4,t-1}$, while $y_{3,t-1}$ is irrelevant. Each variable does not directly depend on its own past; indeed, the main diagonal contains zeros. This setting mimics a stylized telemetry scenario where only a small subset of cross-metric lagged effects is present, while all lagged metrics are included as candidate predictors. $\square$

4.2. Baseline estimation via LASSO and dependency graph interpretation

A standard approach to estimate Eq. (2) under sparsity is the LASSO [58]:

$$\hat{\boldsymbol{\alpha}}_i = \arg \min_{\boldsymbol{\alpha}_i \in \mathbb{R}^{np}} \left\{ \frac{1}{2T} \|\mathbf{y}_i - \mathbf{X}' \boldsymbol{\alpha}_i\|_2^2 + \lambda \|\boldsymbol{\alpha}_i\|_1 \right\}, \quad (4)$$

where $\lambda > 0$ controls the strength of regularization and is selected via a tuning criterion. The ℓ_1 penalty produces sparse solutions that can be mapped to a directed dependency graph: a non-zero coefficient on a lagged predictor is interpreted as a predictive (Granger-type) relation from one metric to another (see, e.g., [6]).

Example 2. Figure 1 shows the graph for $y_{1,t}$ in Example 1. We can see that directed arrows indicate non-zero coefficients in the VAR coefficient matrix $\mathbf{A}$. In particular, the arrows from $y_{2,t-1}$ and $y_{4,t-1}$ to $y_{1,t}$ represent the coefficients $\alpha_{1,2} \neq 0$ and $\alpha_{1,4} \neq 0$, respectively. This implies that the past values of y_2 and y_4 have a direct influence on the evolution of y_1 in the VAR system.

Conversely, the absence of an arrow from $y_{3,t-1}$ to $y_{1,t}$ indicates that the corresponding coefficient is zero, implying that the past values of y_3 do not contribute to predicting y_1 . The graph therefore, illustrates the *sparsity structure* of the model: only a subset of lagged variables act as relevant predictors. $\square$

This interpretability is attractive in systems engineering workflows (e.g., performance diagnosis and capacity planning), but it also makes *false positives* particularly harmful, because they introduce spurious edges that may mislead practitioners. In the context of LASSO regression, a false positive refers to a variable that is assigned a nonzero coefficient by the estimator even though its true underlying coefficient is zero (i.e., $\hat{\alpha}_{i,j} \neq 0$ when $\alpha_{i,j} = 0$ for the j th equation). This typically arises due to sampling variability, noise, or correlations among predictors, which can lead the procedure to incorrectly select irrelevant variables. In the context of workload modeling, this translates into spurious cross-metric dependencies and misleading interpretations of the inferred dependency graph.

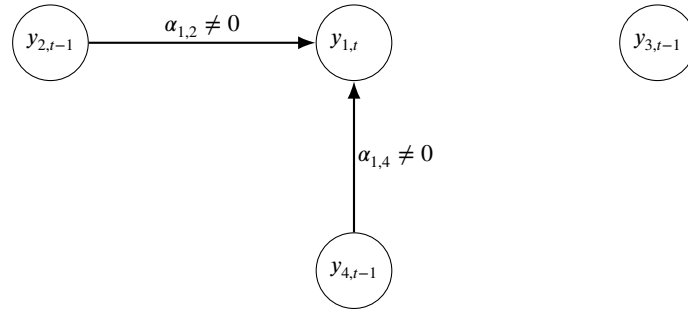


Figure 1: Sparse VAR interpreted as a directed dependency graph for the variable $y_{1,t}$ in the illustrative example.

4.3. Why serial correlation can create spurious dependencies

LASSO variable selection is known to degrade when predictors are strongly correlated, because correlated designs make it difficult to separate relevant from irrelevant variables and can undermine conditions required for stable support recovery [71, 7, 8, 51]. In time series, correlation arises not only from true cross-metric coupling, but also from *serial dependence* that reduces the effective number of independent observations [59]. As a result, even modest persistence can inflate sample correlations among lagged predictors, increasing the chance that irrelevant predictors appear informative and are incorrectly selected. A similar issue arises in system telemetry, where many variables exhibit serial correlation, further amplifying spurious associations among predictors.

Irrepresentable condition and selection stability The reliability of LASSO variable selection depends on structural properties of the predictor correlation matrix. A key requirement is the *irrepresentable condition* (IC) [71, 8], which informally states that irrelevant predictors should not be too strongly correlated with the relevant ones. In particular, let S denote the index set of relevant predictors and S^C its complement. The IC requires:

$$\left| (\mathbf{X})_{S^C} (\mathbf{X})_S' ((\mathbf{X})_S (\mathbf{X})_S')^{-1} \text{Sign}(\boldsymbol{\alpha}^*(S)) \right| < \mathbf{1} - \varphi,$$

where $\varphi \in (0, 1)$ and the inequality holds element-wise. Hereafter, we refer to the quantity $(\mathbf{X})_{S^C} (\mathbf{X})_S' ((\mathbf{X})_S (\mathbf{X})_S')^{-1}$, which summarizes how strongly irrelevant predictors can be represented by relevant ones, as the *representability value*.

This condition constrains the correlation between irrelevant and relevant predictors. If violated, sign consistency and support recovery of the LASSO estimator may fail.

When the IC is violated, LASSO may fail to recover the correct sparsity pattern, resulting in unstable coefficient estimates and increased false positives. In the context of workload modeling, this translates into spurious cross-metric dependencies and misleading interpretations of the inferred dependency graph.

Serial correlation indirectly affects the IC by inflating correlations among lagged predictors, thereby increasing the probability of its violation.

To capture this operationally, we allow each disturbance process to exhibit short-memory dynamics:

$$\epsilon_{i,t} = \sum_{l=1}^{p_\epsilon} \phi_{\epsilon,l} \epsilon_{i,t-l} + \omega_{i,t}, \quad (5)$$

where $\omega_{i,t}$ is zero-mean white noise with finite variance and is uncorrelated across components and non-zero lags. This models a common situation in monitored systems: residual persistence remains even after accounting for lagged cross-metric effects (e.g., due to buffering, batching, periodic background jobs, or unmodeled control loops).

When the innovation process exhibits serial dependence, persistence propagates through the VAR recursion and alters the population covariance structure of the lagged predictors. As a result, variables that are not directly linked through the VAR coefficient matrices may nonetheless display non-zero population correlations (see, e.g., [27], Ch. 3 and Ch. 10; [43], Ch. 2 and Ch. 11).

Example 3. In our illustrative example, we note that when ϕ increases, persistence in ϵ_t propagates through the VAR recursion and inflates correlations among the lagged predictors. With reference to our running example, it can be shown that the irrelevant predictor $y_{3,t-1}$ depicted in Figure 1, becomes increasingly correlated with the relevant predictors as ϕ grows. In graph terms, this would correspond to adding a spurious edge in Figure 1 from node $y_{3,t-1}$ to $y_{1,t}$. Here we provide details on this aspect. Let $\mathbf{X}$ denote the matrix of lagged predictors excluding lags of the response $y_{1,t}$, i.e., $\mathbf{X}$ contains $(y_{2,t-1}, y_{3,t-1}, y_{4,t-1})$. Under Eq. (3), the population correlation matrix is:

$$\boldsymbol{\Sigma}_x^{(1)} = E[\mathbf{X}\mathbf{X}'] = \begin{pmatrix} 1 & \rho_{23} & 0 \\ \rho_{23} & 1 & \rho_{34} \\ 0 & \rho_{34} & 1 \end{pmatrix},$$

with

$$\rho_{23} = \frac{\alpha\phi(\alpha\phi + 1)}{\sqrt{2\alpha^4 + 2\alpha^2 + 1 + 2\alpha^3\phi}},$$

$$\rho_{34} = \frac{\alpha^2\phi^2}{\sqrt{2\alpha^4 + 2\alpha^2 + 1 + 2\alpha^3\phi}}.$$

In our illustrative example we have $S = \{y_{2,t-1}, y_{4,t-1}\}$ and $S^C = \{y_{3,t-1}\}$. Consequently, in terms of representability values, we have

$$(\mathbf{X})_{S^C}(\mathbf{X})'_S ((\mathbf{X})_S(\mathbf{X})'_S)^{-1} = \left(\frac{\alpha\phi(\alpha\phi + 1)}{\sqrt{2\alpha^4 + 2\alpha^2 + 1 + 2\alpha^3\phi}}, \frac{\alpha^2\phi^2}{\sqrt{2\alpha^4 + 2\alpha^2 + 1 + 2\alpha^3\phi}} \right).$$

These expressions show that persistence ($|\phi|$) increases correlations between relevant and irrelevant predictors. $\square$

Example 3 makes this effect explicit: even though $y_{3,t-1}$ is irrelevant in the VAR representation, it becomes correlated in population with the relevant predictors due to persistence in the innovations. This highlights that, in the presence of serially correlated disturbances, zero entries in the VAR coefficient matrices do not imply independence among lagged predictors.

4.4. Proposed estimator: ARMAR-LASSO for sparse VARs

To mitigate spurious dependencies induced by serial correlation, we build on the ARMAR-LASSO idea [59] and adapt it to sparse VAR estimation. The key principle is *pre-whitening*: before applying LASSO, each predictor time series is filtered to reduce serial correlation, so that correlations among predictors reflect cross-metric dependence rather than persistence-driven artifacts.

ARMAR-LASSO is implemented as a two-step procedure:

1. Pre-whitening of predictors. For each metric series used as a predictor, we fit a finite-order univariate ARMA model and extract residuals (innovation-like components). These residuals are substantially less serially correlated than the original series.
2. Sparse regression on pre-whitened predictors (plus response lags). We run a LASSO regression using the pre-whitened residuals as predictors. In addition, we include lags of the response metric to capture self-dependence and improve forecasting accuracy, as commonly done to reduce residual autocorrelation in time-series regression [32] and as suggested for ARMAR-LASSO in [59].

The expected effect is twofold. First, by reducing persistence in predictors, the method lowers spurious sample correlations and decreases the chance that irrelevant predictors appear strongly associated with the response. Second, by keeping the final model sparse, it preserves interpretability: the non-zero coefficients still provide a compact set of predictive cross-metric relations.

The following example shows that ARMAR-LASSO mitigates this risk by replacing each predictor series with its ARMA residuals. Since these residuals are serially uncorrelated, irrelevant predictors are less likely to “shadow” relevant ones and be erroneously selected into the model.

Example 4. After fitting ARMA models and using residuals as predictors, the residual predictor matrix $\mathbf{U}$ satisfies:

$$\Sigma_u^{(1)} = E[\mathbf{U}\mathbf{U}'] = \mathbf{I}_3,$$

i.e., the relevant and irrelevant predictors become uncorrelated in this toy setting. In particular, in terms of representability values, we have, for pre-whitened predictors,

$$(\mathbf{U})_{S^C}(\mathbf{U})'_S ((\mathbf{U})_S(\mathbf{U})'_S)^{-1} = (0, 0).$$

Figure 2 provides an empirical illustration of this effect by comparing representability value. While this value increases with ϕ for the original predictors, it remains near zero after pre-whitening, aligning with the intuition that ARMAR-LASSO stabilizes variable selection under serial dependence.

As discussed above, representability values are directly related to the irrerepresentable condition, and therefore provide a useful diagnostic of potential LASSO selection failures.

Practical takeaway For workload modeling, this matters because spurious selections do not only degrade prediction; they can also lead to misleading interpretations (e.g., false cross-metric causal narratives) and misdirect root-cause analysis. ARMAR-LASSO aims to preserve the interpretability benefits of sparse VARs while reducing persistence-driven false discoveries.

In Section 5, we empirically study these effects through Monte Carlo experiments under varying serial dependence and cross-metric multicollinearity. $\square$

5. Monte Carlo Experiments

In this section, we evaluate the two aspects of the adopted approach, namely performance in terms of RMSE, and its ability to detect irrelevant variables. This is studied using synthetically generated data across varying sample sizes T , degree of multicollinearity in the correlation matrix of error terms (Σ_ω), and parameter values ϕ , which control the degree of serial correlation.

We generate a VAR(1) as follows: $\mathbf{y}_t = \mathbf{A}\mathbf{y}_{t-1} + \epsilon_t$, $t = 1, \dots, T$, $n = 10$,

$$\mathbf{A} = \begin{pmatrix} 0.0 & 0.3 & 0.3 & 0.0 & 0.3 & 0.3 & 0.0 & 0.0 & 0.3 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.3 & 0.3 & 0.0 & 0.3 & 0.0 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.0 & 0.3 \\ 0.3 & 0.0 & 0.3 & 0.3 & 0.0 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 \\ 0.0 & 0.3 & 0.0 & 0.0 & 0.3 & 0.0 & 0.0 & 0.0 & 0.3 & 0.3 \\ 0.3 & 0.3 & 0.3 & 0.0 & 0.0 & 0.3 & 0.0 & 0.0 & 0.3 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \end{pmatrix},$$

$$\epsilon_{i,t} = \phi\epsilon_{i,t-1} + \omega_{i,t},$$

where $\omega_t \sim MVN(\mathbf{0}_{10}, \Sigma_\omega)$. We consider two different scenarios for Σ_ω : *Orthogonal scenario*: $\Sigma_\omega = \mathbf{I}_{10}$. *Multicollinear scenario*: $\Sigma_\omega = \mathbf{W}_{10}$, that is, the covariance matrix has a Wishart distribution with 10 degrees of freedom. Figure 3 reports Σ_ω in the multicollinear scenario.

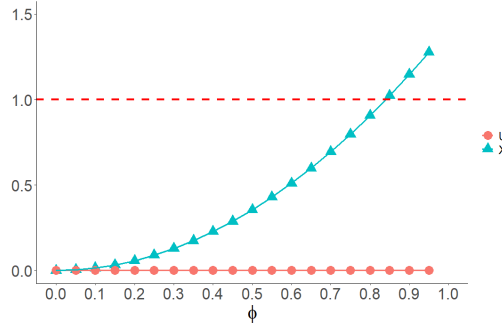


Figure 2: Illustrative example. Representability values computed on the original lagged predictors ($\mathbf{X}$) and on the ARMA pre-whitened residual predictors ($\mathbf{U}$) for $\phi = 0.0, 0.05, 0.1, \dots, 0.95$ (with $\alpha = 2$ and $\sigma^2 = 1$). The red dashed line marks the threshold below which the irrepresentable condition is satisfied. Serial correlation increases representability in the original predictor space, while pre-whitening keeps it close to zero, reducing the risk of persistence-driven false positives.

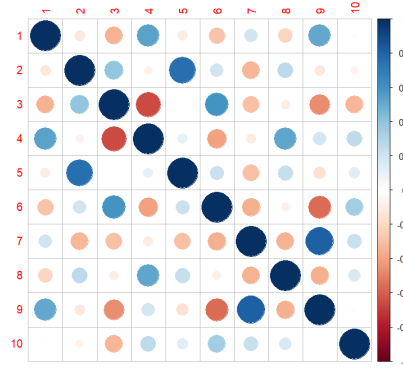


Figure 3: Covariance matrix Σ_{ω} in the multicollinear scenario used in the simulation study.

We compare ARMAr-LASSO with the standard LASSO applied to the observed time series plus two lags of y_t . The performance of each method is evaluated based on average results from 1000 independent simulations, focusing on the Root Mean Square Forecast Error (RMSE) and precision. In the next section, we further assess the interpretability of the estimated models by analyzing their ability to recover the underlying dependency structure. Simulations have varying sample size, $T = 100, 1000$ and $\phi = 0.3, 0.6, 0.9$. For both methods, the tuning parameter λ is selected using the Bayesian Information Criterion (BIC). For the ARMAr-LASSO, estimated residuals (the $\hat{u}$'s) are obtained filtering each time series with an ARMA(p_i, q_i), where p_i and q_i are selected by BIC within $0 \leq p_i, q_i \leq 2, i = 1, \dots, n$.

In terms of performance metrics, we use various indicators of accuracy concerning both variable selection and forecasting accuracy, namely: Precision, $Prec = TP / (TP + FP)$, where TP indicates true positives and FP indicates false positives; the representability value, RV defined as $(\mathbf{X})_{S_i}^c (\mathbf{X})_{S_i}' (\mathbf{X})_{S_i} (\mathbf{X})_{S_i}'^{-1}$ for the observed time series, and $(\hat{\mathbf{U}})_{S_i}^c (\hat{\mathbf{U}})_{S_i}' (\hat{\mathbf{U}})_{S_i} (\hat{\mathbf{U}})_{S_i}'^{-1}$ for the estimated ARMA residuals; the Root Mean Square Forecast Error, $RMSE = \sqrt{\frac{1}{nT} \sum_{i=1}^n \sum_{t=1}^T (\hat{y}_{i,t} - y_{i,t})^2}$.

Figure 4 plots (a) and (b) report Precision. In the orthogonal scenario, Precision increases noticeably when the sample size grows from $T = 100$ (plot (a)) to $T = 1000$ (plot (b)). This improvement reflects the fact that some false positives are mainly driven by sampling variability and tend to disappear as more observations become available. In larger samples, the methods can more clearly distinguish between relevant and irrelevant predictors, leading to a cleaner recovery of the true sparse structure. In contrast, this pattern is much less pronounced in the multicollinear case, where the covariance matrix of the innovations is generated from a Wishart distribution. Here, the presence of strong cross-correlations among regressors makes variable selection intrinsically more difficult, so that increasing T does not fully eliminate spurious inclusions. Nevertheless, across all configurations (independently of the sample size T and the degree of multicollinearity in Σ_{ω}) ARMAr-LASSO consistently achieves higher Precision than standard LASSO, indicating a lower number of false positives. Moreover, the advantage of ARMAr-LASSO becomes more pronounced as ϕ increases, i.e., as the degree of serial correlation in the innovations becomes stronger. This pattern is consistent with the mechanism illustrated in the *Illustrative Example*, where filtering the series helps isolate the relevant dynamic relationships.

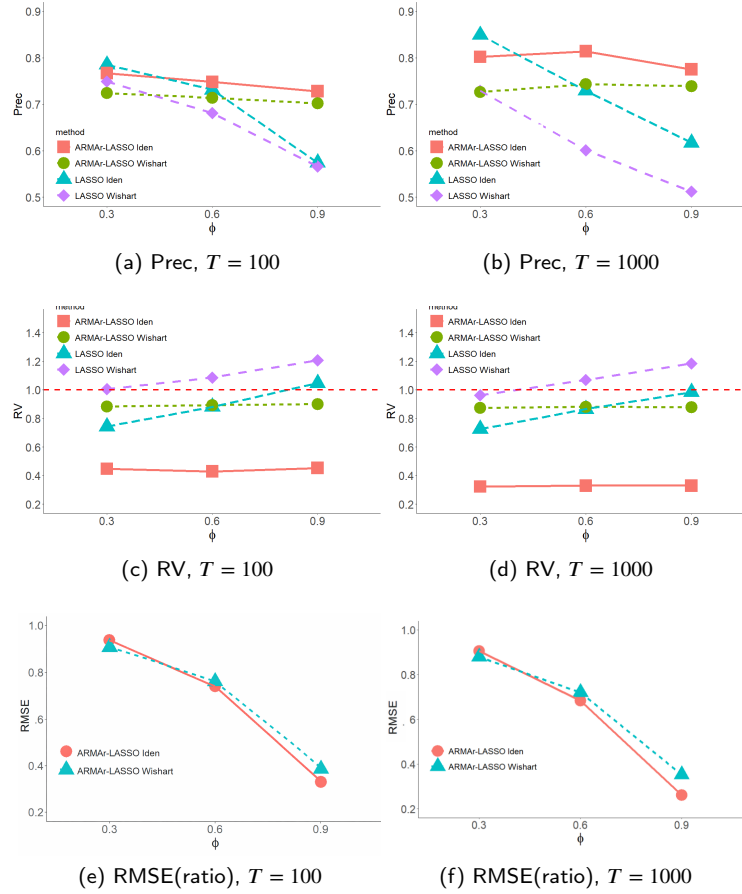


Figure 4: Monte Carlo comparison of LASSO and ARMAR-LASSO. Plots (a)–(b) show Precision, highlighting improved performance with larger T under orthogonality and consistent false-positive reduction by ARMAR-LASSO, especially as ϕ increases. Plots (c)–(d) report Representability Value (RV), which is higher under Wishart covariance but systematically lower with ARMAR-LASSO. Plots (e)–(f) display RMSE ratios relative to LASSO, indicating forecast gains under stronger serial correlation.

Figure 4 plots (c) and (d) report the representability value (RV). As expected, when Σ_ω is drawn from the Wishart distribution shown in Figure 3, the RV is systematically higher than in the identity case. This reflects the stronger dependence structure among the regressors, which increases the probability of violating the representability condition required for correct variable selection with LASSO. However, for any given generating distribution of Σ_ω , ARMAR-LASSO consistently produces lower RV values than the standard LASSO. This indicates that filtering the series reduces the effective correlation structure among predictors and therefore mitigates the conditions that hinder sparse recovery. In contrast, for the standard LASSO the RV tends to increase with ϕ , showing that stronger serial correlation exacerbates the representability problem when the data are used without preliminary filtering. These results reinforce the intuition provided by the illustrative example.

Finally, Figure 4 plots (e) and (f) display the RMSE relative to LASSO. The results show that the forecasting performance of ARMAR-LASSO improves progressively as ϕ increases. In particular, the RMSE ratio steadily declines, indicating that ARMAR-LASSO produces more accurate forecasts than LASSO when the innovations exhibit stronger

serial correlation. This finding is consistent with the idea that filtering the series before applying the LASSO reduces the impact of serial dependence and leads to more reliable coefficient estimates, which ultimately translate into improved predictive performance.

5.1. Interpretability analysis under serial dependence

A key challenge in assessing interpretability in real-world datasets is the lack of a known ground-truth dependency structure. In particular, for the datasets considered in Section 6 (Kubernetes, Exathlon, and Materna), the underlying network is not observable, making it difficult to evaluate the accuracy of the inferred relationships.

To address this limitation, we use synthetic data to compare the true network with the estimated ones. This approach is standard in the literature on time series network estimation (see, e.g., [6]) and allows us to assess the ability of different methods to recover the true sparsity pattern.

We examine how serial dependence affects interpretability, focusing on the recovery of the sparsity pattern of the

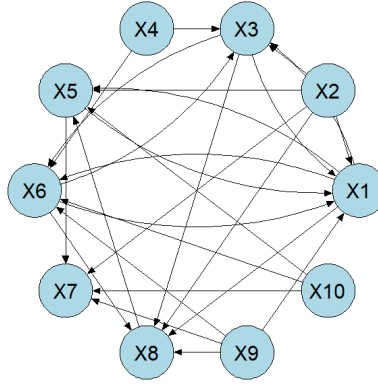


Figure 5: Ground-truth dependency structure (gold standard network). Edges correspond to non-zero coefficients in the VAR model.

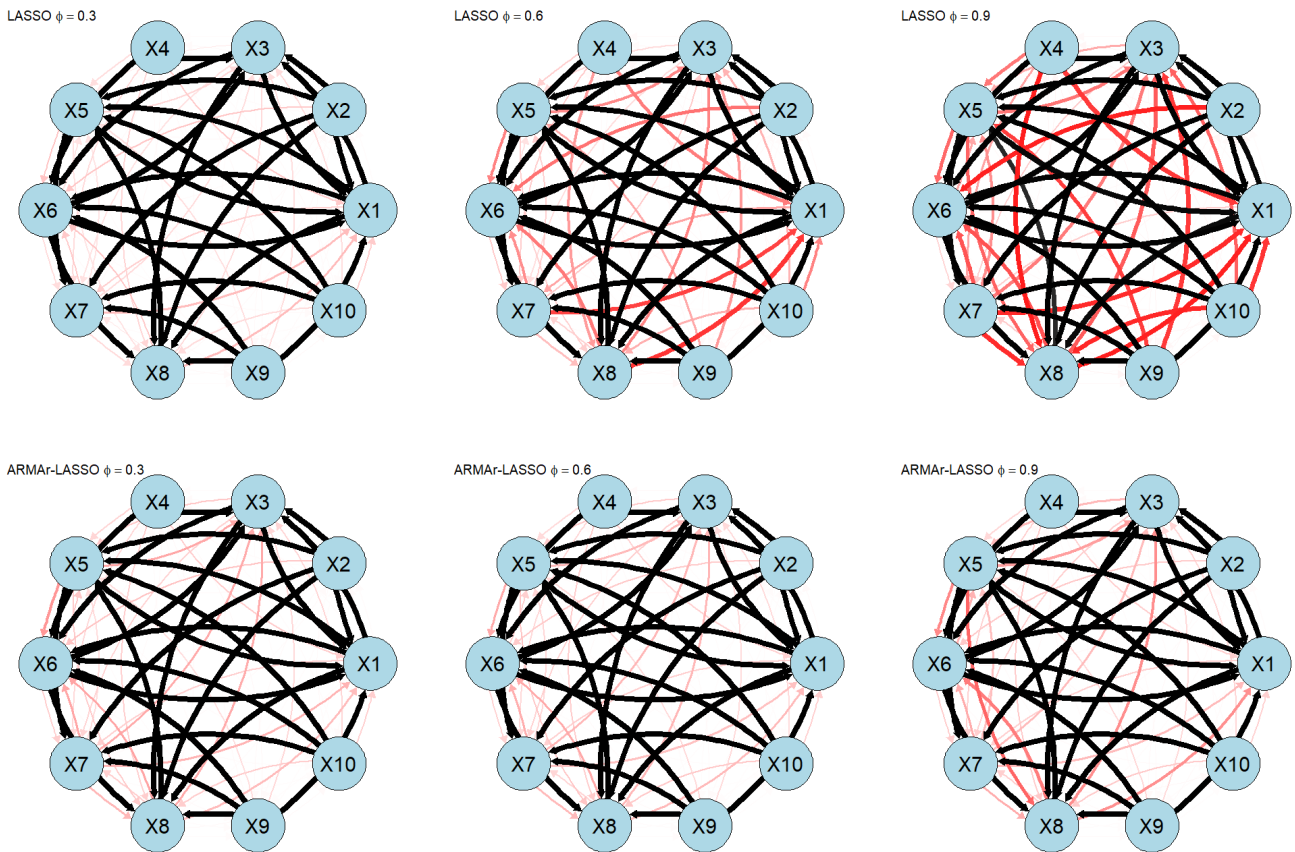


Figure 6: Estimated dependency networks for $T = 1000$ when the irrepresentable condition (IC) holds. Black and red edges denote, respectively, true positives (edges matching the ones in Figure 5) and false positives (spurious additional edges). The intensities of the colors of the edges denote the frequency with which they were selected in the simulations. As the sample size increases, LASSO selects an increasing number of false positive edges due to persistence-induced correlations, while ARMAR-LASSO remains stable.

VAR coefficient matrices under varying levels of persistence. Figure 5 shows the ground-truth network, where edges correspond to non-zero coefficients in the VAR model.

We then estimate the dependency networks using both LASSO and ARMAR-LASSO under increasing levels of serial correlation and for different sample sizes. Figures 6 and 7 report the corresponding results for $T = 1000$ for the IC-satisfied and IC-violated settings, respectively. The

results for $T = 100$ are reported in [Appendix A](#) (Figures 10 and 11).

For $T = 1000$, a clear pattern emerges. When the irrepresentable condition holds (Figure 6), LASSO performs well in low-serial-correlation regimes, recovering most of the true edges with few false positives. However, as the level of serial correlation increases, the number of spurious

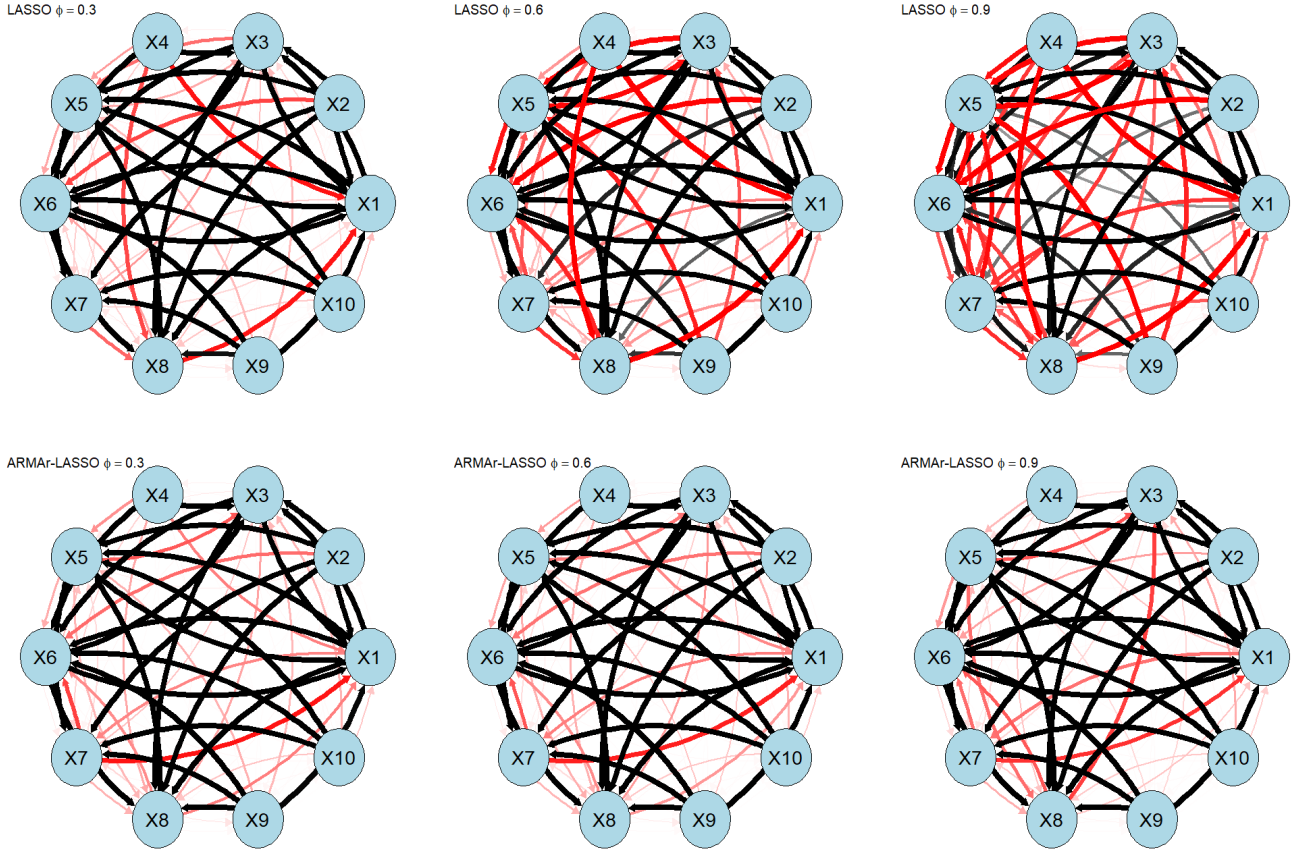


Figure 7: Estimated dependency networks for $T = 1000$ when the irrepresentable condition (IC) is violated. The number of false positive edges selected by LASSO further increases with larger sample size, while ARMAR-LASSO continues to recover a sparse structure consistent with the ground truth.

edges grows, leading to increasingly dense and less interpretable networks. In contrast, ARMAR-LASSO remains stable across different levels of persistence, consistently recovering a sparse structure close to the ground truth.

When the IC is violated (Figure 7), LASSO exhibits a large number of false positives already at moderate levels of serial correlation, and the inferred networks quickly become unreliable. ARMAR-LASSO, on the other hand, remains substantially more robust, with the estimated networks preserving a sparse structure and a closer agreement with the gold standard.

These results are consistent with the analysis in Section 4: the spurious edges are not solely a finite-sample artifact, but are partly driven by population-level correlations induced by serial dependence. In fact, a large sample size does not eliminate these effects and may even make them more apparent. In contrast, ARMAR-LASSO remains stable for large T , as the pre-whitening step reduces persistence-driven correlations among predictors.

These results highlight that serial correlation does not only affect predictive performance, but can severely distort the inferred dependency structure. By mitigating persistence-driven correlations, ARMAR-LASSO improves the interpretability of sparse VAR models, leading to more reliable network reconstruction.

6. Experimental Evaluation

This section assesses the effectiveness of the proposed ARMAR-LASSO approach in terms of forecasting accuracy, robustness, and parsimony. We consider multiple experimental scenarios designed to reflect common workload prediction settings, including fixed-sample evaluation, rolling-window forecasting, and multi-step prediction with forecasted regressors.

6.1. Setup

We consider five datasets, including both cloud workload traces and multivariate system metrics. The first contains CPU and RAM usage of individual pods in a Kubernetes cluster [9], where the workload traces are generated following [63]. The second dataset is the Materna Workload Traces (GWA-T-13), widely used [50, 3] in cloud-computing research for workload modeling and resource prediction¹. The remaining three datasets are drawn from the Exathlon collection [30], commonly employed in industrial fault-detection studies. The selection of these datasets is motivated by the need for multivariate time series comprising several distinct and heterogeneous variables (e.g., Kubernetes cluster Figure 9), which excludes commonly such as Azure [15], Google

¹<https://www.kaggle.com/datasets/kpiyush04/maternaworkloadtraces>

Cluster [53], and Alibaba [25] which are characterized usually by only memory and cpu utilization.

In all experiments, ARMAR-LASSO filters each predictor using an ARMA(p, q) model with orders restricted to $p, q \leq 5$. Both ARMAR-LASSO and LASSO include five lags of the target variable as additional predictors. Prediction accuracy is evaluated using the Root Mean Squared Error (RMSE). We compare ARMAR-LASSO against: standard LASSO, ARMA models, and state-of-the-art machine learning and deep learning baselines implemented using the Darts library [28]. More in detail, we consider DLinear [69], Transformer [36], NHITS [12], and TiDE [17] for deep learning, Random forest [54] as a representative of the traditional machine learning methods. Unless otherwise specified, model hyperparameters follow the default settings provided by the respective libraries. We consider three experimental settings: fixed training–test split, rolling-window forecasting, and multi-step prediction with forecasted regressors. These settings allow us to evaluate the proposed method under progressively more demanding forecasting scenarios. We consider three standard performance metrics: RMSE, which measures the average magnitude of forecast errors in the original scale of the data and penalizes large errors more heavily, SMAPE [29], which measures relative forecast accuracy as a symmetric percentage error and is less scale-dependent than RMSE, and MASE [45], which measures forecast accuracy relative to a naive benchmark, allowing errors to be compared across series with different scales.

We deliberately refrain from extensive hyperparameter tuning for the machine learning and deep learning baselines, and instead adopt their default configurations as provided by the respective libraries. This choice reflects a practitioner-oriented perspective, where models are often deployed under limited time and computational budgets, and extensive tuning may not be feasible. Our goal is therefore to assess the relative performance of different approaches in a realistic setting, rather than under idealized, fully optimized conditions. We also focus our comparison on sparse linear models, as interpretability is a central requirement in workload modeling. For this reason, we do not include Elastic Net, as our focus is on maximal sparsity and interpretability of the inferred dependency graph. Elastic Net, by combining ℓ_1 and ℓ_2 penalties, tends to favor stability in the presence of correlated predictors rather than minimal variable selection, often leading to less sparse and less interpretable solutions. Similarly, we do not consider variants such as Adaptive LASSO, as they introduce additional hyperparameters and modeling choices that would require further tuning. Consistently with our practitioner-oriented approach, we instead restrict attention to standard LASSO, which provides a well-established and widely used baseline with minimal tuning requirements.

6.2. Fixed training–test split

The first setting follows the standard forecasting framework with a fixed training sample and an out-of-sample test

sample. For the Kubernetes dataset, we adopt the training and test splits provided by the authors on the corresponding GitHub repository². Specifically, the training set consists of 961 observations, while the test set comprises 480 observations. For the remaining datasets, we use 4000 observations for Materna and 1600 observations for the Exathlon datasets, allocating 70% of the data to the training set and the remaining 30% to the test set. In this setting, ARMAR-LASSO is compared with the standard LASSO, and with the above-mentioned techniques from Darts.

Table 1 reports the results for this setting. We consider the Kubernetes, Materna, and Exathlon datasets, and evaluate the competing methods in terms of RMSE, SMAPE, MASE, and computational time.

In terms of RMSE, ARMAR-LASSO and LASSO provide highly competitive results across all datasets. On the Kubernetes dataset, ARMAR-LASSO achieves the lowest average RMSE, while LASSO ranks as the second-best method. A similar pattern is observed on the Materna dataset, where ARMAR-LASSO again provides the best average performance. Across the Exathlon datasets, ARMAR-LASSO remains competitive, although DLinear achieves the lowest RMSE on Exathlon Data 1 and Exathlon Data 3. On Exathlon Data 2, ARMAR-LASSO provides the best average RMSE among the considered methods.

The results based on SMAPE and MASE further confirm the strong performance of the regularized regression approaches. For SMAPE, ARMAR-LASSO is either the best or among the best-performing methods in most datasets, often closely followed by LASSO. The main exception is the Kubernetes dataset, where TiDE achieves the lowest average SMAPE. For MASE, ARMAR-LASSO is again consistently competitive, being the best or second-best method across the datasets. These results indicate that the good performance of ARMAR-LASSO is not limited to scale-dependent metrics such as RMSE, but also persists under relative and scaled accuracy measures. The large MASE values observed for some Exathlon datasets are mainly due to distributional changes between training and test periods: the training series are nearly stationary, whereas the test series exhibit stronger trends and larger fluctuations. Consequently, even moderate test-set forecast errors can yield large MASE values.

Concerning runtime, ARMAR-LASSO and LASSO are consistently more computationally efficient than the considered deep-learning benchmarks. Standard LASSO is slightly faster than ARMAR-LASSO, since the latter requires an additional ARMA-based pre-whitening step. This overhead becomes more pronounced in high-dimensional settings, such as Kubernetes, where a larger number of variables must be filtered. Nevertheless, ARMAR-LASSO remains substantially cheaper than deep-learning models, which require significantly longer training times.

Overall, these results convey a clear practical message: on these datasets and across different metrics, regularized regression methods, namely ARMAR-LASSO and LASSO,

²<https://github.com/JoseLuisC99/predictive-pod-resource-utilization/tree/main/data>

Table 1

RMSE, MAE, SMAPE and MASE metrics on the Exathlon, Kubernetes and Materna datasets. Green and blue cells denote the best and second-best results, respectively.

Methods	ARMAR-LASSO	LASSO	TiDE	Transformer	DLinear	NHITS	RF
Exathlon Data 1							
AVG. MAE	7.452	8.017	9.436	25.679	6.061	8.162	23.150
AVG. RMSE	8.890	9.280	11.194	27.251	7.418	9.520	24.846
AVG. SMAPE	63.134	58.448	85.293	181.312	67.315	77.961	127.090
AVG. MASE	36.831	42.922	79.703	266.527	47.866	79.901	249.353
RUNTIME(s.)	4.098	2.396	32.966	86.415	22.829	37.028	66.959
Exathlon Data 2							
AVG. MAE	1.109	1.356	1.242	1.738	1.761	1.522	1.672
AVG. RMSE	1.4963	1.757	1.6310	2.1648	2.2560	1.9661	2.0776
AVG. SMAPE	47.820	57.048	56.003	62.760	71.839	62.711	61.512
AVG. MASE	8.445	9.593	9.750	22.108	10.202	16.855	26.148
RUNTIME(s.)	4.427	2.333	33.395	90.685	23.579	38.426	69.879
Exathlon Data 3							
AVG. MAE	36.480	38.354	58.670	40.394	34.892	39.323	40.144
AVG. RMSE	41.055	42.776	66.499	45.638	40.466	44.630	45.380
AVG. SMAPE	129.448	141.163	153.558	170.536	141.284	151.940	171.696
AVG. MASE	132.161	137.650	295.183	645.989	126.849	337.257	647.890
RUNTIME(s.)	4.662	2.143	33.496	86.351	22.881	37.496	67.418
Kubernetes							
AVG. MAE	0.337	0.357	0.812	0.999	0.470	0.817	1.042
AVG. RMSE	0.554	0.581	1.300	1.627	0.783	1.306	1.671
AVG. SMAPE	52.111	55.674	88.192	107.785	65.174	93.461	116.853
AVG. MASE	1.295	1.424	3.205	4.089	1.931	3.347	3.992
RUNTIME(s.)	22.792	2.301	32.721	79.520	22.916	36.444	68.604
Materna							
AVG. MAE	0.470	0.549	0.528	0.924	0.599	0.632	0.745
AVG. RMSE	0.939	1.039	1.018	1.667	1.085	1.256	1.239
AVG. SMAPE	96.894	102.740	79.566	109.651	97.295	84.932	99.322
AVG. MASE	1.266	1.397	2.108	16.125	1.741	3.620	2.798
RUNTIME(s.)	4.612	2.233	89.056	238.347	57.537	92.675	169.662

often outperform or closely match more sophisticated deep-learning architectures such as Transformer, NHITS, TiDE, and DLinear when forecasting accuracy and computational cost are jointly considered. In our experiments, the additional complexity of deep-learning models does not consistently translate into superior forecasting performance.

The cases in which deep-learning methods are competitive are mainly observed for RMSE on Exathlon Data 1 and Exathlon Data 3, where DLinear achieves the best performance. This behavior may be related to distributional changes in the Exathlon datasets: while the training data are largely stationary, the test sets exhibit stronger trend components and larger fluctuations. Models such as DLinear, which are designed to capture low-frequency patterns, may therefore be better suited to handle these forms of non-stationarity. Notably, DLinear is the simplest among the considered Darts models. Indeed, previous work has shown that DLinear, a one-layer linear model, can outperform more complex architectures in several forecasting tasks [69]. This finding is consistent with the broader message of our analysis: simple and well-regularized methods can outperform more complex models.

Nevertheless, even when DLinear achieves the lowest RMSE on some datasets, or TiDE obtains the best SMAPE on Kubernetes, the performance gains over ARMAR-LASSO

remain relatively small. These modest improvements come at a substantially higher computational cost. This trade-off is especially relevant in rolling-window and online forecasting settings, where models must be repeatedly re-estimated and computational efficiency becomes a key requirement.

In contrast, ARMAR-LASSO provides competitive predictive performance while maintaining a low computational cost and producing sparse, interpretable models. These properties are especially valuable in workload management scenarios, where both timely predictions and clear insights into cross-metric dependencies are essential.

6.3. Rolling Window

In the second setting, we adopt a rolling-window forecasting scheme with window size 200 and forecast horizons $h = 1, 2, 3, 4$. At each iteration, the model is trained on the most recent 200 observations and used to produce forecasts at horizons 1 through 4. All experiments in this setting are conducted exclusively on the Kubernetes dataset, as it provides the largest number of variables (22) and the most heterogeneous multivariate structure, making it particularly suitable for evaluating the ability of ARMAR-LASSO to achieve parsimonious variable selection and improved interpretability. As benchmarks, we include both a standard ARMA(p, q) model—commonly adopted in rolling-window

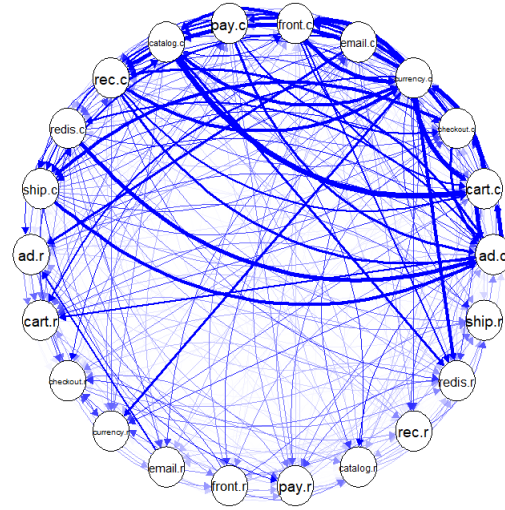
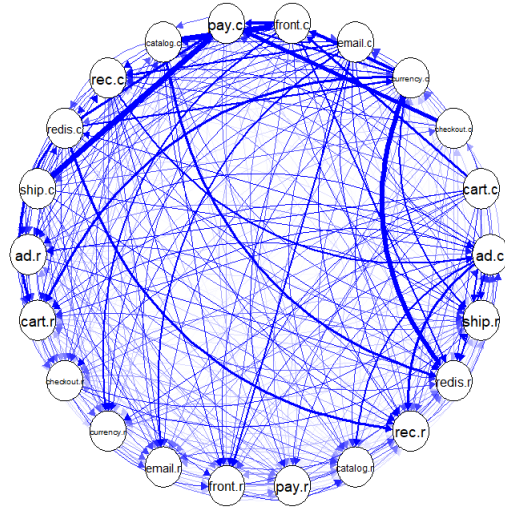


Figure 8: Dependency networks estimated by LASSO (left) and ARMAR-LASSO (right) on the Kubernetes dataset. ARMAR-LASSO yields a sparser and more interpretable structure.

forecasting contexts—where, for each variable, the orders are restricted to $p, q \leq 5$ and selected using the Bayesian Information Criterion (BIC), and the LASSO approach. In this case, deep learning methods are not considered, as their training process cannot be executed at each rolling-window shift within a reasonable computational time.

Table 2 summarizes the rolling-window results. In this setting, ARMAR-LASSO is systematically more accurate than LASSO in terms of mean MAE, RMSE, SMAPE and MASE across all horizons. ARMA is competitive on SMAPE and MASE metric for h equal to 1, but his performance deteriorates also with the respect to LASSO when the horizon h increases.

Beyond accuracy, the key difference between the two regularized methods concerns parsimony. ARMAR-LASSO selects on average about 60 regressors, whereas LASSO selects more than 90 (see the “Avg. Var.” rows in Table 2). This reduction in dimensionality is also clearly visible in Figure 9: the LASSO heatmap is noticeably denser, indicating a more widespread and less selective inclusion of predictors, while ARMAR-LASSO produces a sparser structure, concentrating selection on a smaller set of consistently relevant relationships.

Figure 8 reports the networks obtained by LASSO and ARMAR-LASSO. The two networks highlight a clear difference in sparsity between the methods. The LASSO network appears considerably denser, with a large number of edges connecting most nodes, which makes the overall dependency structure difficult to interpret. In contrast, the ARMAR-LASSO network is noticeably sparser, with fewer and more prominent edges, resulting in a cleaner and more readable structure.

From a performance-evaluation and system-workload perspective, the differences between heatmaps in Figure 9

reveal how variable selection directly affects both predictive efficiency and interpretability. The denser heatmap produced by the standard LASSO (Figure 9) indicates that a large number of workload metrics are repeatedly selected across rolling windows. This behavior is largely driven by strong serial dependence and cross-metric correlations (e.g., between CPU- and memory-related signals), leading to limited gains in forecasting accuracy and reduced clarity from an operational standpoint.

In contrast, ARMAR-LASSO yields a sparser and more stable selection pattern (Figure 9), concentrating on a smaller subset of variables that are consistently informative for prediction. These variables typically correspond to metrics that reflect sustained resource pressure or systematic cross-service interactions, such as CPU-intensive components acting as workload drivers or memory-related signals that integrate longer-term demand effects. From a workload-management perspective, this parsimonious representation is particularly valuable: it improves forecasting robustness, limits error propagation in rolling-window settings, and provides clearer insights into which components and resources most strongly influence overall system performance.

6.4. Rolling Window with Predicted Regressor

The third setting further increases the difficulty by using predicted regressors for multi-step horizons ($h = 2, 3, 4$). In this scenario, forecast errors are no longer confined to the target variable but propagate through the covariates, potentially amplifying uncertainty and substantially degrading predictive performance. As a result, model complexity and variable redundancy play a critical role, making parsimony a key requirement for reliable multi-step forecasting.

Consistently with the findings in the Rolling Window setting, ARMAR-LASSO selects a substantially smaller set

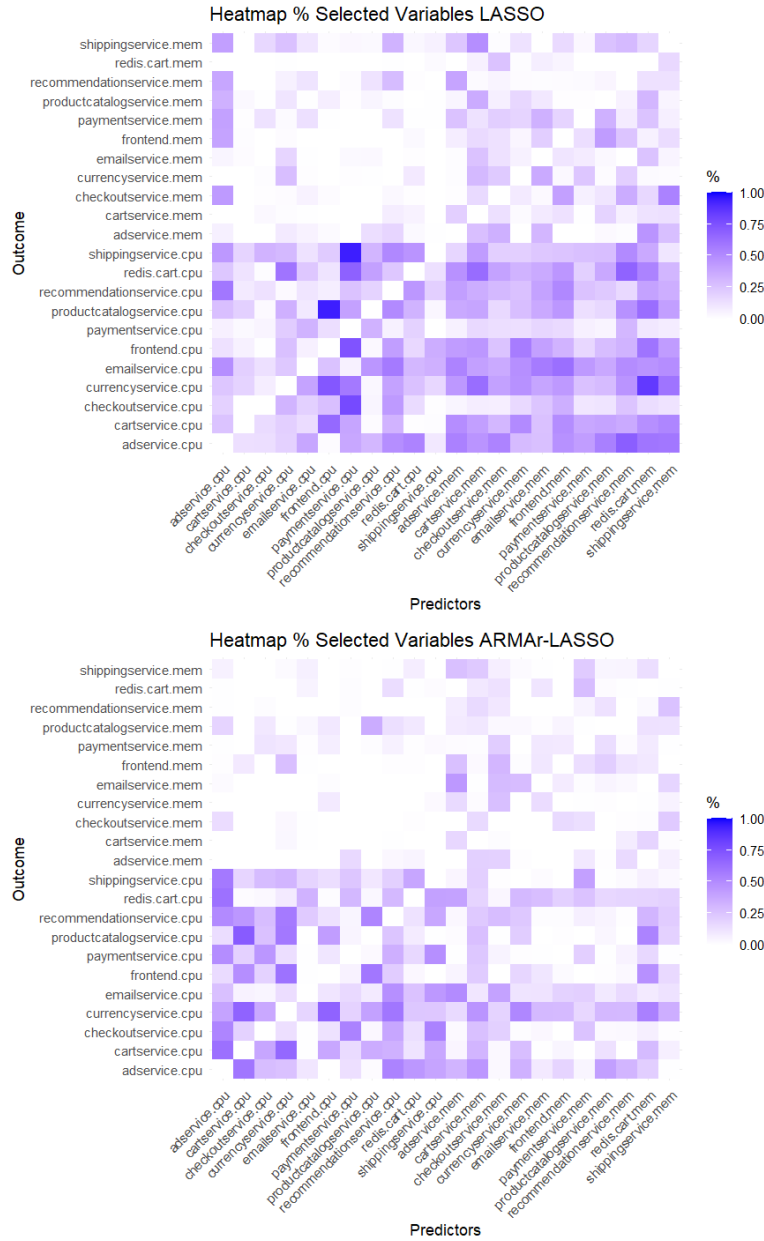


Figure 9: Heatmap of the selection frequency (%) for LASSO in the rolling-window setting (Kubernetes). Darker cells indicate predictors more frequently selected across windows.

of predictors than the standard LASSO. This characteristic becomes a concrete advantage when regressors are forecasted, as relying on fewer covariates effectively limits the channels through which prediction errors can propagate over time. By concentrating on a reduced set of consistently informative variables, ARMAR-LASSO produces forecasts that are more stable and less sensitive to compounding errors across horizons.

This effect is clearly reflected in Table 3, where the relative MAE, RMSE, SMAPE e MASE of ARMAR-LASSO steadily improves against LASSO as the forecast horizon increases.

Taken together, Settings 2 and 3 demonstrate that ARMAR-LASSO enhances predictive accuracy, and also

yields more stable and interpretable forecasting models. These properties are particularly valuable in demanding operational settings, such as workload and resource management systems, where covariates must be forecasted and robustness to error propagation is essential.

7. Conclusions and Future Work

This paper studied sparse regularized methods for workload prediction in complex software systems based on operational telemetry data. Focusing on LASSO-based sparse VAR models, we analyzed how serial correlation in system telemetry affects both variable selection and forecasting

Table 2

RMSE, MAE, SMAPE and MASE metrics on the Kubernetes dataset for rolling window of size 200 and forecasting horizon $h \in \{1, 2, 3, 4\}$. Green and blue cells denote the best and second-best results, respectively. The Avg. Var. entries of ARMA contain NA, not applicable, because ARMA is univariate and therefore does not perform any variable selection.

Horizon	Metric	ARMAR-LASSO	LASSO	ARMA
$h = 1$	AVG. MAE	0.321	0.338	0.321
	AVG. RMSE	0.557	0.634	0.622
	AVG. SMAPE	52.061	53.288	50.721
	AVG. MASE	1.215	1.289	1.214
	AVG. VAR.	60.740	95.067	NA
$h = 2$	AVG. MAE	0.455	0.471	0.486
	AVG. RMSE	0.838	0.891	0.946
	AVG. SMAPE	64.327	65.366	66.695
	AVG. MASE	1.776	1.849	1.908
	AVG. VAR.	60.656	94.825	NA
$h = 3$	AVG. MAE	0.577	0.589	0.609
	AVG. RMSE	1.068	1.098	1.126
	AVG. SMAPE	73.509	74.292	78.833
	AVG. MASE	2.316	2.360	2.439
	AVG. VAR.	60.402	94.867	NA
$h = 4$	AVG. MAE	0.682	0.694	0.714
	AVG. RMSE	1.245	1.274	1.289
	AVG. SMAPE	81.057	81.808	89.412
	AVG. MASE	2.783	2.836	2.911
	AVG. VAR.	60.515	94.742	NA

Table 3

RMSE, MAE, SMAPE and MASE metrics of ARMAR-LASSO and LASSO across horizons in the rolling-window setting with predicted regressors (Kubernetes). Green cells denote the best results. The increasing advantage at longer horizons reflects reduced error propagation due to the more parsimonious ARMAR-LASSO specification.

Horizon	Metric	ARMAR-LASSO	LASSO
$h = 1$	AVG. MAE	0.320	0.345
	AVG. RMSE	0.601	0.642
	AVG. SMAPE	51.053	54.967
	AVG. MASE	1.217	1.319
$h = 2$	AVG. MAE	0.537	0.613
	AVG. RMSE	1.070	1.290
	AVG. SMAPE	71.110	78.125
	AVG. MASE	2.195	2.527
$h = 3$	AVG. MAE	0.751	0.983
	AVG. RMSE	1.627	2.729
	AVG. SMAPE	85.655	93.689
	AVG. MASE	3.170	4.156
$h = 4$	AVG. MAE	0.974	1.364
	AVG. RMSE	2.529	4.333
	AVG. SMAPE	96.500	105.225
	AVG. MASE	4.203	5.977

performance. We showed that, in multivariate settings, persistence can induce spurious cross-metric dependencies that undermine predictive accuracy and interpretability, leading to misleading dependency structures. To address this issue, we proposed the ARMAR-LASSO framework, a LASSO-based method for multivariate workload modeling that incorporates a pre-whitening step prior to sparse regression. Through illustrative example, Monte Carlo simulations, and extensive experiments on real-world benchmark datasets, we demonstrated that ARMAR-LASSO consistently improves forecasting accuracy while selecting substantially fewer predictors than standard LASSO. These gains are

observed across fixed-sample evaluation, rolling-window forecasting, and multi-step prediction with forecasted regressors—settings that are particularly relevant in operational environments. We further compared ARMAR-LASSO with state-of-the-art deep learning approaches, showing that the proposed method can match or outperform these more complex models at a fraction of the computational cost. This makes ARMAR-LASSO especially suitable for scenarios where models must be frequently re-estimated or deployed under strict resource constraints. Beyond predictive performance, our results highlight the practical value of parsimonious and explainable models for software performance engineering. By reducing persistence-driven false positives, ARMAR-LASSO yields more stable and interpretable dependency structures, facilitating clearer diagnostic insights and more reliable performance analysis.

Future work will explore extensions to non-stationary and regime-switching workloads, where both temporal dynamics and dependency structures evolve over time. Another promising direction is the integration of sparse dependency models into hybrid forecasting and control frameworks, enabling tighter coupling between prediction, system adaptation, and resource management while preserving interpretability and computational efficiency.

References

- [1] Amiri, M., Mohammad-Khanli, L., 2017. Survey on prediction models of applications for resources provisioning in cloud. *Journal of Network and Computer Applications* 82, 93–113.
- [2] Arunarani, A., Manjula, D., Sugumaran, V., 2019. Task scheduling techniques in cloud computing: A literature survey. *Future Generation Computer Systems* 91, 407–415.
- [3] Ba, A., 2024. Future workload and cloud resource usage: Insights from an interpretable forecasting model, in: 2024 IEEE International

- Conference on Big Data (BigData), IEEE. pp. 2283–2287.
- [4] Bao, J., Lu, Z., Wu, J., Zhang, S., Zhong, Y., 2014. Implementing a novel load-aware auto scale scheme for private cloud resource management platform, in: 2014 IEEE Network Operations and Management Symposium (NOMS), IEEE. pp. 1–4.
 - [5] Barati, M., Sharifian, S., 2015. A hybrid heuristic-based tuned support vector regression model for cloud load prediction. *The Journal of Supercomputing* 71, 4235–4259.
 - [6] Barigozzi, M., Brownlee, C., 2019. Nets: Network estimation for time series. *Journal of Applied Econometrics* 34, 347–364. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jae.2676>, doi:<https://doi.org/10.1002/jae.2676>, arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/jae.2676>.
 - [7] Bickel, P.J., Ritov, Y., Tsybakov, A.B., 2009. Simultaneous analysis of lasso and dantzig selector. *The Annals of Statistics* 37, 1705–1732. URL: <http://dx.doi.org/10.1214/08-AOS620>, doi:10.1214/08-aos620.
 - [8] Bühlmann, P., van de Geer, S., 2011. *Statistics for high-dimensional data*. Springer Series in Statistics, Springer, Heidelberg. URL: <http://dx.doi.org/10.1007/978-3-642-20192-9>, doi:10.1007/978-3-642-20192-9. methods, theory and applications.
 - [9] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J., 2016. Borg, omega, and kubernetes. *Commun. ACM* 59, 50–57. URL: <https://doi.org/10.1145/2890784>, doi:10.1145/2890784.
 - [10] Calheiros, R.N., Masoumi, E., Ranjan, R., Buyya, R., 2014. Workload prediction using arima model and its impact on cloud applications' qos. *IEEE transactions on cloud computing* 3, 449–458.
 - [11] Cetinski, K., Juric, M.B., 2015. Ame-wpc: Advanced model for efficient workload prediction in the cloud. *Journal of Network and Computer Applications* 55, 191–201.
 - [12] Challu, C., Olivares, K.G., Oreshkin, B.N., Ramirez, F.G., Canseco, M.M., Dubrawski, A., 2023. Nhits: Neural hierarchical interpolation for time series forecasting, in: *Proceedings of the AAAI conference on artificial intelligence*, pp. 6989–6997.
 - [13] Chen, Z., Hu, J., Min, G., Zomaya, A.Y., El-Ghazawi, T., 2019. Towards accurate prediction for high-dimensional and highly-variable cloud workloads with deep learning. *IEEE Transactions on Parallel and Distributed Systems* 31, 923–934.
 - [14] Chieu, T.C., Mohindra, A., Karve, A.A., Segal, A., 2009. Dynamic scaling of web applications in a virtualized cloud computing environment, in: 2009 IEEE International Conference on e-Business Engineering, IEEE. pp. 281–286.
 - [15] Cortez, E., Bonde, A., Muzio, A., Russinovich, M., Fontoura, M., Bianchini, R., 2017. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms, in: *Proceedings of the 26th Symposium on Operating Systems Principles*, pp. 153–167.
 - [16] Das, A., Kong, W., Leach, A., Mathur, S., Sen, R., Yu, R., 2023a. Long-term forecasting with tide: Time-series dense encoder. *arXiv preprint arXiv:2304.08424*.
 - [17] Das, A., Kong, W., Leach, A., Mathur, S., Sen, R., Yu, R., 2023b. Long-term forecasting with tide: Time-series dense encoder. *arXiv preprint arXiv:2304.08424*.
 - [18] Davis, R.A., Zang, P., Zheng, T., 2016. Sparse vector autoregressive modeling. *Journal of Computational and Graphical Statistics* 25, 1077–1096.
 - [19] Devi, K.L., Valli, S., 2023. Time series-based workload prediction using the statistical hybrid model for the cloud environment. *Computing* 105, 353–374.
 - [20] Durango-Cohen, P.L., 2007. A time series analysis framework for transportation infrastructure management. *Transportation Research Part B: Methodological* 41, 493–505.
 - [21] Ekambaram, V., Jati, A., Nguyen, N., Sinthong, P., Kalagnanam, J., 2023. Tsmixer: Lightweight mlp-mixer model for multivariate time series forecasting, in: *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, pp. 459–469.
 - [22] Feng, B., Ding, Z., 2023. Group: An end-to-end multi-step-ahead workload prediction approach focusing on workload group behavior, in: *Proceedings of the ACM Web Conference 2023*, pp. 3098–3108.
 - [23] Feng, B., Ding, Z., Jiang, C., 2026. Mgroup: Multi-instance workload prediction approach based on group behavior perception. *IEEE Transactions on Parallel and Distributed Systems*.
 - [24] Garbi, G., Incerto, E., Tribastone, M., 2023. μp : A development framework for predicting performance of microservices by design, in: 2023 IEEE 16th International Conference on Cloud Computing (CLOUD), IEEE. pp. 178–188.
 - [25] Guo, J., Chang, Z., Wang, S., Ding, H., Feng, Y., Mao, L., Bao, Y., 2019. Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces, in: *Proceedings of the international symposium on quality of service*, pp. 1–10.
 - [26] Gupta, S., Dinesh, D.A., 2017. Resource usage prediction of cloud workloads using deep bidirectional long short term memory networks, in: 2017 IEEE international conference on advanced networks and telecommunications systems (ANTS), IEEE. pp. 1–6.
 - [27] Hamilton, J.D., 1994. *Time Series Analysis*. Princeton University Press. URL: <http://www.jstor.org/stable/j.ctv14jx6sm>.
 - [28] Herzen, J., Lässig, F., Piazzetta, S.G., Neuer, T., Tafti, L., Raille, G., Pottelbergh, T.V., Pasieka, M., Skrodzki, A., Huguenin, N., Dumonal, M., Kościusz, J., Bader, D., Gusset, F., Benheddi, M., Williamson, C., Kosinski, M., Petrik, M., Grosch, G., 2022. Darts: User-friendly modern machine learning for time series. *Journal of Machine Learning Research* 23, 1–6. URL: <http://jmlr.org/papers/v23/21-1177.html>.
 - [29] Hyndman, R.J., Koehler, A.B., 2006. Another look at measures of forecast accuracy. *International journal of forecasting* 22, 679–688.
 - [30] Jacob, V., Song, F., Stiegler, A., Rad, B., Diao, Y., Tatbul, N., 2020. Exathlon: A benchmark for explainable anomaly detection over time series. *arXiv:2010.05073*.
 - [31] James, G., Witten, D., Hastie, T., Tibshirani, R., 2013. An introduction to statistical learning: with applications in r URL: <https://faculty.marshall.usc.edu/gareth-james/ISL/>.
 - [32] Keele, L., Kelly, N.J., 2006. Dynamic models for dynamic theories: The ins and outs of lagged dependent variables. *Political Analysis* 14, 186–205. URL: https://EconPapers.repec.org/RePEc:cup:polals:v:14:y:2006:i:02:p:186-205_00.
 - [33] Khan, A., Yan, X., Tao, S., Anerousis, N., 2012. Workload characterization and prediction in the cloud: A multiple time series approach, in: 2012 IEEE Network Operations and Management Symposium, IEEE. pp. 1287–1294.
 - [34] Khan, T., Tian, W., Ilager, S., Buyya, R., 2022. Workload forecasting and energy state estimation in cloud data centres: MI-centric approach. *Future Generation Computer Systems* 128, 320–332.
 - [35] Khorsand, R., Ghobaei-Arani, M., Ramezanpour, M., 2018. With-drawn: a fuzzy auto-scaling approach using workload prediction for mmog application in a cloud environment.
 - [36] Kim, T., Kim, J., Tae, Y., Park, C., Choi, J.H., Choo, J., 2021. Reversible instance normalization for accurate time-series forecasting against distribution shift, in: *International conference on learning representations*.
 - [37] Kousiouris, G., Cucinotta, T., Varvarigou, T., 2011. The effects of scheduling, workload type and consolidation scenarios on virtual machine performance and their prediction through optimized artificial neural networks. *Journal of Systems and Software* 84, 1270–1291.
 - [38] Li, S., Wang, Y., Qiu, X., Wang, D., Wang, L., 2013. A workload prediction-based multi-vm provisioning mechanism in cloud computing, in: 2013 15th Asia-Pacific network operations and management symposium (APNOMS), IEEE. pp. 1–6.
 - [39] Liu, M., Zeng, A., Chen, M., Xu, Z., Lai, Q., Ma, L., Xu, Q., 2022. Scinet: Time series modeling and forecasting with sample convolution and interaction. *Advances in neural information processing systems* 35, 5816–5828.
 - [40] Lu, Y., Yu, X., Li, S., Zheng, H., Cui, J., Zhong, H., Liu, L., Min, G., 2025. Stiff-net: Spatial-temporal insights via image-driven feature fusion for workload prediction in intelligent cloud data centers. *IEEE Transactions on Cloud Computing* 13, 1385–1396.
 - [41] Luo, Y., Gao, M., Yu, Z., Ge, H., Gao, X., Cai, T., Chen, G., 2024. Integrating system state into spatio temporal graph neural network for microservice workload prediction, in: *Proceedings of the 30th ACM*

- SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 5521–5531.
- [42] Luo, Y., Wang, Y., Long, L., Cai, T., Gao, X., Chen, G., 2026. Steps: State-aware spatio-temporal workload distribution prediction for cloud service scaling. *IEEE Transactions on Services Computing*.
 - [43] Lütkepohl, H., 2005. New introduction to multiple time series analysis. Springer, Berlin [u.a.]. URL: http://gso.gbv.de/DB=2.1/CMD?ACT=SRCHA&SRT=YOP&IKT=1016&TRM=ppn+366296310&sourceid=fbw_bibsonomy.
 - [44] Mahdhi, T., Mezni, H., 2018. A prediction-based vm consolidation approach in iaas cloud data centers. *Journal of Systems and Software* 146, 263–285.
 - [45] Makridakis, S., 1993. Accuracy measures: theoretical and practical concerns. *International journal of forecasting* 9, 527–529.
 - [46] Makridakis, S., Spiliotis, E., Assimakopoulos, V., 2018. Statistical and machine learning forecasting methods: Concerns and ways forward. *PloS one* 13, e0194889.
 - [47] Masdari, M., Khoshnevis, A., 2020. A survey and classification of the workload forecasting methods in cloud computing. *Cluster Computing* 23, 2399–2424.
 - [48] Minarolli, D., Freisleben, B., 2014. Cross-correlation prediction of resource demand for virtual machine resource allocation in clouds, in: 2014 Sixth International Conference on Computational Intelligence, Communication Systems and Networks, pp. 119–124. doi:10.1109/CICSyN.2014.36.
 - [49] Mishra, S.K., Sahoo, B., Parida, P.P., 2020. Load balancing in cloud computing: a big picture. *Journal of King Saud University-Computer and Information Sciences* 32, 149–158.
 - [50] Naveen Kumar, M., Annappa, B., Yadav, V., 2025. Efficient kalman filter based deep learning approaches for workload prediction in cloud and edge environments. *Computing* 107, 10.
 - [51] Negahban, S.N., Ravikumar, P., Wainwright, M.J., Yu, B., 2012. A unified framework for high-dimensional analysis of m -estimators with decomposable regularizers. *Statistical Science* 27. URL: <http://dx.doi.org/10.1214/12-STS400>, doi:10.1214/12-sts400.
 - [52] Patel, Y.S., Misra, R., 2018. Performance comparison of deep vm workload prediction approaches for cloud, in: Progress in Computing, Analytics and Networking: Proceedings of ICCAN 2017. Springer, pp. 149–160.
 - [53] Reiss, C., Wilkes, J., Hellerstein, J.L., 2011. Google cluster-usage traces: format+ schema. Google Inc., White Paper 1, 83.
 - [54] Rigatti, S.J., 2017. Random forest. *Journal of insurance medicine* 47, 31–39.
 - [55] Schmidl, S., Wenig, P., Papenbrock, T., 2022. Anomaly detection in time series: a comprehensive evaluation. *Proc. VLDB Endow.* 15, 1779–1797. URL: <https://doi.org/10.14778/3538598.3538602>, doi:10.14778/3538598.3538602.
 - [56] Seshadri, K., Sindhu, K., Bhattu, S.N., Kollengode, C., 2024. Design and evaluation of a hierarchical characterization and adaptive prediction model for cloud workloads. *IEEE Transactions on Cloud Computing* 12, 712–724.
 - [57] Tarsa, S.J., Kumar, A.P., Kung, H., 2014. Workload prediction for adaptive power scaling using deep learning, in: 2014 IEEE International Conference on IC Design & Technology, IEEE. pp. 1–5.
 - [58] Tibshirani, R., 1996. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B* 58, 267–288.
 - [59] Tonini, S., Chiaromonte, F., Giovannelli, A., 2026a. Armar-lasso: Mitigating the impact of predictor serial correlation on the lasso. URL: <https://arxiv.org/abs/2408.09288>, arXiv:2408.09288.
 - [60] Tonini, S., Vandin, A., Chiaromonte, F., Licari, D., Barsacchi, F., 2026b. Fast, robust, and accurate anomaly detection for multivariate time series. *Advances in Data Analysis and Classification* URL: <https://doi.org/10.1007/s11634-026-00667-8>, doi:10.1007/s11634-026-00667-8.
 - [61] Tsay, R.S., 2005. Analysis of financial time series. John Wiley & sons.
 - [62] Tuli, S., Gill, S.S., Garraghan, P., Buyya, R., Casale, G., Jennings, N.R., 2021. Start: Straggler prediction and mitigation for cloud computing environments using encoder lstm networks. *IEEE Transactions on Services Computing* 16, 615–627.
 - [63] Turin, G., Borgarelli, A., Donetti, S., Damiani, F., Johnsen, E.B., Tapia Tarifa, S.L., 2023. Predicting resource consumption of kubernetes container systems using resource models. *Journal of Systems and Software* 203, 111750. URL: <https://www.sciencedirect.com/science/article/pii/S0164121223001450>, doi:https://doi.org/10.1016/j.jss.2023.111750.
 - [64] Wang, H., Peng, J., Huang, F., Wang, J., Chen, J., Xiao, Y., 2023. Micn: Multi-scale local and global context modeling for long-term series forecasting, in: The eleventh international conference on learning representations.
 - [65] Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., Long, M., 2022. Timesnet: Temporal 2d-variation modeling for general time series analysis. arXiv preprint arXiv:2210.02186.
 - [66] Wu, H., Xu, J., Wang, J., Long, M., 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems* 34, 22419–22430.
 - [67] Xu, M., Song, C., Wu, H., Gill, S.S., Ye, K., Xu, C., 2022. esdnn: deep neural network based multivariate workload prediction in cloud computing environments. *ACM Transactions on Internet Technology (TOIT)* 22, 1–24.
 - [68] Yekta, M., Shahhoseini, H.S., 2023. A review on machine learning methods for workload prediction in cloud computing, in: 2023 13th International Conference on Computer and Knowledge Engineering (ICCKE), IEEE. pp. 306–311.
 - [69] Zeng, A., Chen, M., Zhang, L., Xu, Q., 2023. Are transformers effective for time series forecasting?, in: Proceedings of the AAAI conference on artificial intelligence, pp. 11121–11128.
 - [70] Zhan, Z.H., Liu, X.F., Gong, Y.J., Zhang, J., Chung, H.S.H., Li, Y., 2015. Cloud computing resource scheduling and a survey of its evolutionary approaches. *ACM Computing Surveys (CSUR)* 47, 1–33.
 - [71] Zhao, P., Yu, B., 2006. On model selection consistency of lasso. *The Journal of Machine Learning Research* 7, 2541–2563. URL: <http://dl.acm.org/citation.cfm?id=1248547.1248637>.
 - [72] Zhong, W., Zhuang, Y., Sun, J., Gu, J., 2018. A load prediction model for cloud computing using pso-based weighted wavelet support vector machine: W. zhong et al. *Applied Intelligence* 48, 4072–4083.
 - [73] Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., Zhang, W., 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting, in: Proceedings of the AAAI conference on artificial intelligence, pp. 11106–11115.
 - [74] Zhou, T., Ma, Z., Wen, Q., Wang, X., Sun, L., Jin, R., 2022. Fed-former: Frequency enhanced decomposed transformer for long-term series forecasting, in: International conference on machine learning, PMLR. pp. 27268–27286.

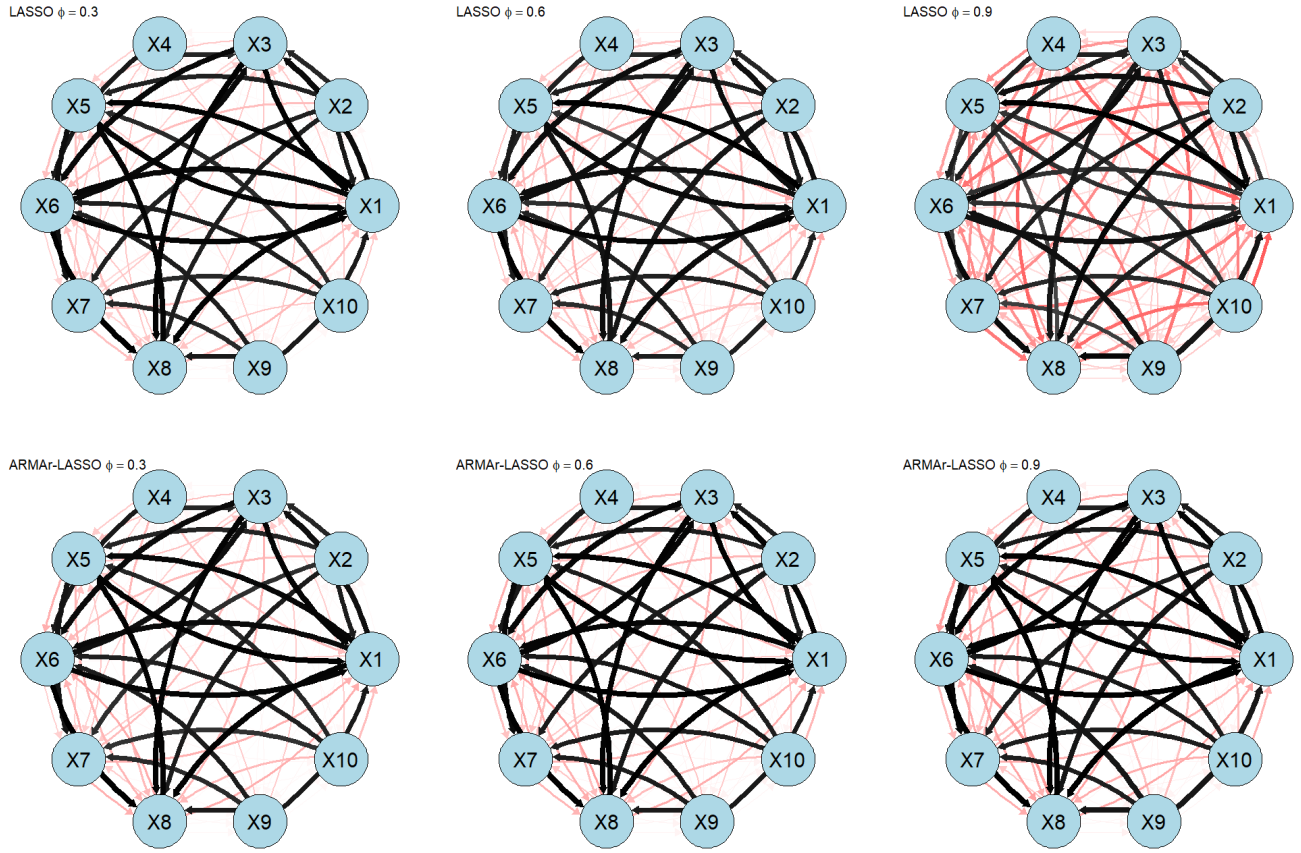


Figure 10: Estimated dependency networks for $T = 100$ when the irrerepresentable condition (IC) holds, under increasing levels of serial correlation. LASSO progressively selects more false positive edges as persistence increases, while ARMAR-LASSO remains stable and recovers a sparse structure close to the ground truth.

A. Estimated dependency networks for $T = 100$

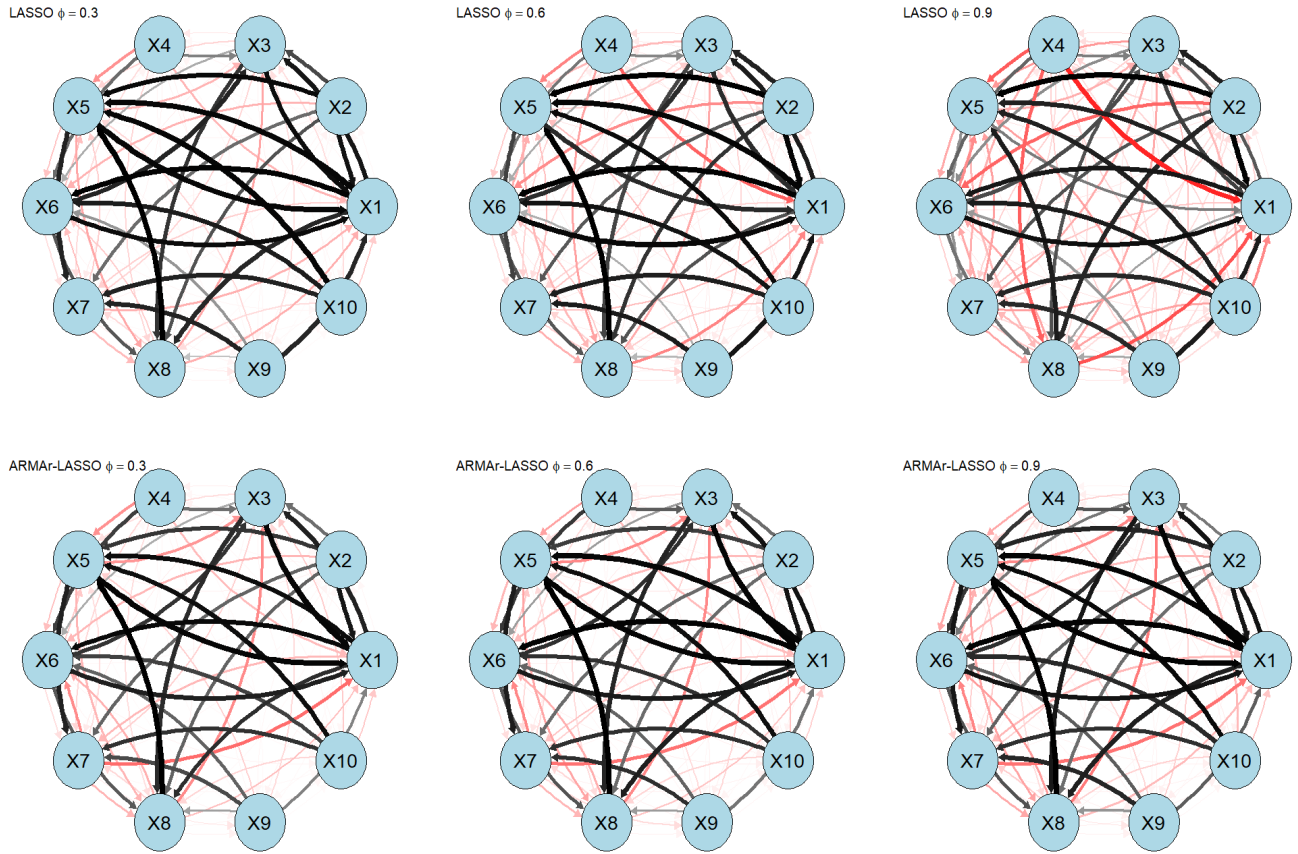


Figure 11: Estimated dependency networks for $T = 100$ when the irrerepresentable condition (IC) is violated. LASSO exhibits a large number of false positive edges even at moderate levels of serial correlation, leading to dense and less interpretable networks. ARMAR-LASSO remains more robust and preserves a sparse structure.